Memoria de Prácticas

Introducción al Aprendizaje Automático

Jesús Fernández López
Rafael Carlos Schoettel Vilchez

Marzo, 2026

Capítulo 1

Cuaderno de Laboratorio — Práctica 3: Metodología de Evaluación y Rigor Científico

Objetivo

Aprender a realizar evaluaciones honestas en condiciones difíciles, especialmente con datos desbalanceados, y detectar errores metodológicos graves como la fuga de datos (*data leakage*).

Material de partida

Se parte de:

- Un dataset sintético de **1000 pacientes** con **5 variables predictoras**.
- Una clase positiva muy minoritaria (**2 %** de los casos), es decir, $K = 20$ de $N = 1000$.

1.1. Tarea 1 — La lotería de la partición aleatoria

Configuración común

Modelo usado: `LogisticRegression` (con `class_weight='balanced'`)
Métricas usadas: `Nº positivos en test`, `% clase 1`
Semilla(s): `0, 1, 2, 3, 4` (5 ejecuciones aleatorias)

Hipótesis inicial: Es probable que el número de positivos en test sea muy inestable. Al haber solo un 2 % de casos positivos (20 de 1000), un split aleatorio del 20 % (200 ejemplos) podría coger muy pocos positivos por puro azar, o incluso ninguno. Esto dejaría el test inservible.

1.1.1. Análisis estadístico de la partición

Para comprobar el riesgo real de hacer una mala partición, he calculado la probabilidad usando la distribución hipergeométrica. Teniendo $N = 1000$ pacientes, $K = 20$ positivos en total, y sacando una muestra para test de $n = 200$:

Lo normal (la media esperada) es que caigan 4 positivos en test:

$$\mathbb{E}[X] = \frac{200 \cdot 20}{1000} = 4$$

Pero, ¿qué pasa si tenemos mala suerte? Calculando la probabilidad de que no caiga **ningún** positivo ($X = 0$):

$$P(X = 0) = \frac{\binom{20}{0}\binom{980}{200}}{\binom{1000}{200}} \approx 0{,}0110 \quad (1{,}10\,\%)$$

Si sumamos la probabilidad de sacar 0 o 1 positivo, el riesgo sube a casi un 7 %:

$$P(X \leq 1) \approx 0{,}0110 + 0{,}0563 = 0{,}0673 \quad (6{,}73\,\%)$$

¿Por qué es un problema? Si en el test no hay positivos ($X = 0$), las métricas se rompen:

- **Recall**: No se puede calcular (0/0) porque no hay verdaderos positivos ni falsos negativos reales.
- **F1-score**: Al depender del Recall, también da error.
- **AUC-ROC**: La librería devuelve `NaN` porque necesita ejemplos de ambas clases para pintar la curva.

Conclusión: un mal split aleatorio puede hacer que la evaluación no sirva para nada, sin importar qué modelo usemos.

1.1.2. Registro de resultados (5 particiones aleatorias)

Ejecución	Positivos en test	% clase 1 en test
1	4	2,0 %
2	3	1,5 %
3	5	2,5 %
4	4	2,0 %
5	3	1,5 %

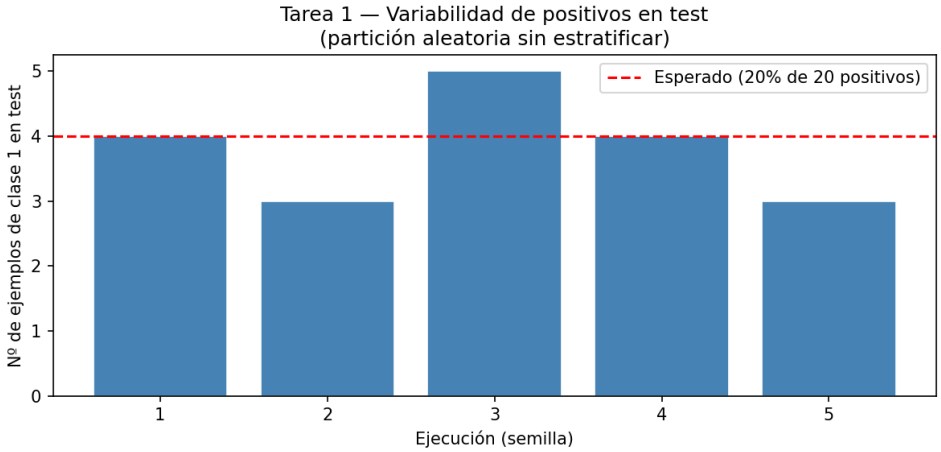

Figura 1.1: Variabilidad del número de ejemplos positivos en el conjunto de test según la semilla. La línea roja indica el valor esperado teórico (4).

Análisis de los resultados:

- *¿Ha cambiado mucho el número de positivos en test entre ejecuciones?* Sí, el número oscila entre 3 y 5 en mis ejecuciones. Aunque numéricamente parezca poco, en un test tan pequeño supone una variación del ±25 %, lo que altera bastante las métricas.

- *¿Sería fiable la evaluación si el azar dejara solo 0 o 1 positivos en test?* Para nada. Como he demostrado antes, hay un ≈ 6,73 % de probabilidad de que ocurra. Con 0 positivos las métricas dan error matemático, y con 1 solo positivo el resultado dependería al 100 % de acertar o fallar en un único paciente, perdiendo todo el rigor estadístico.

1.2. Tarea 2 — Partición estratificada

1.2.1. Registro por folds (`StratifiedKFold`, $k = 10$)

Fold	Positivos en test	Tamaño del fold	Proporción clase 1
1	2	100	2,0 %
2	2	100	2,0 %
3	2	100	2,0 %
4	2	100	2,0 %
5	2	100	2,0 %
6	2	100	2,0 %
7	2	100	2,0 %
8	2	100	2,0 %
9	2	100	2,0 %
10	2	100	2,0 %

Desviación típica de la proporción de clase 1 entre folds: **0,0000 %** (prácticamente nula, lo que confirma que la estratificación funciona correctamente).

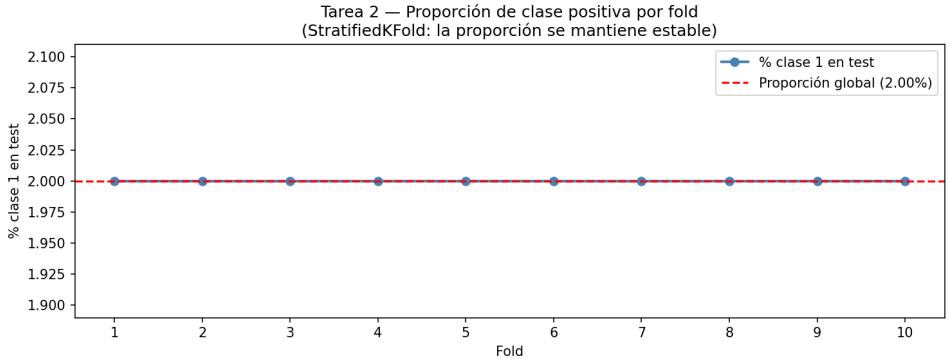

Figura 1.2: Proporción de la clase positiva por fold con `StratifiedKFold`. La línea roja indica la proporción global del dataset (2 %).

Análisis: `StratifiedKFold` garantiza que cada fold mantenga la misma proporción de clases que el dataset original (2 %). En datasets muy desbalanceados, esto evita que algunos folds se queden sin muestras de la clase minoritaria, asegurando que el modelo sea evaluado siempre sobre una distribución representativa y reduciendo la varianza debida al azar.

1.3. Tarea 3 — Comparativa de la varianza

1.3.1. Resumen de resultados (10 iteraciones, 10 folds)

La tabla 1.1 recoge la media y la desviación típica de las tres métricas calculadas mediante `cross_val_score` durante 10 iteraciones independientes de validación cruzada con 10 *folds*. He añadido la métrica AUC-ROC para reforzar las conclusiones sobre la necesidad de estratificación.

Tabla 1.1: Comparativa de estabilidad de métricas: KFold estándar vs. StratifiedKFold (10 iteraciones × 10 folds).

Método	**Accuracy**		**F1-macro**		**AUC-ROC**	
	Media	*std*	Media	*std*	Media	*std*
`KFold`	0,5771	0,0090	0,3829	0,0051	**N/D** (*8/10 iters.*†)	
`StratifiedKFold`	0,5783	**0,0085**	0,3857	**0,0050**	**0,5283**	**0,0253**

† **N/D = No Definida.** En 8 de las 10 iteraciones de `KFold`, algún fold del conjunto de test careció de ejemplos positivos. En esas condiciones, `roc_auc_score` devuelve `NaN` (indefinido), puesto que la curva ROC no puede construirse sin ambas clases representadas. Solo en las 2 iteraciones en que por azar todos los folds tuvieron al menos un positivo se obtuvo un valor numérico (AUC ≈ 0,49). Este resultado es la prueba empírica más directa de la necesidad de estratificación: sin ella, la métrica de evaluación más robusta para clases desbalanceadas queda matemáticamente indefinida en la mayoría de los experimentos.

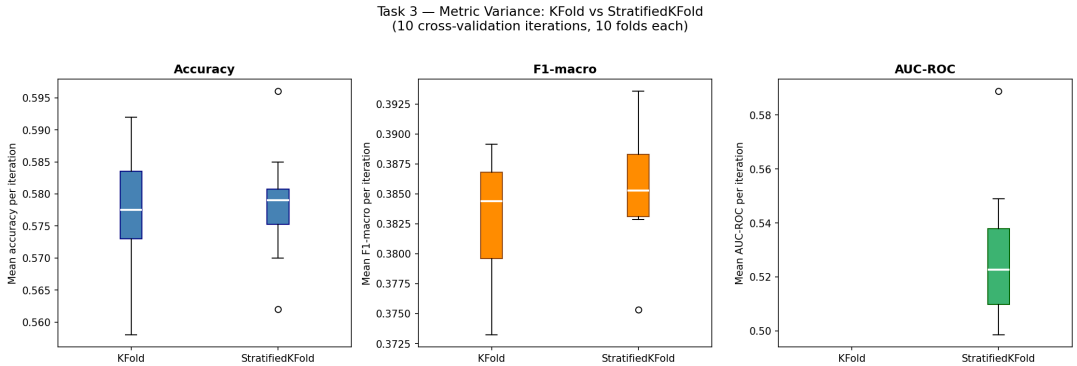

Figura 1.3: Distribución de Accuracy, F1-macro y AUC-ROC a lo largo de 10 iteraciones de validación cruzada (10 *folds*). Cada métrica se muestra como un diagrama de cajas (*boxplot*). `StratifiedKFold` produce cajas más estrechas en las tres métricas, evidenciando menor varianza y mayor estabilidad en la evaluación.

Interpretación:

- *¿Qué método produce métricas más estables?* El método `StratifiedKFold` produce métricas más estables, como se observa en la menor desviación típica (*std*) en Accuracy y F1-macro; pero la diferencia más llamativa se produce en **AUC-ROC**.
- *¿Dónde se nota más la mejora?* En AUC-ROC, sin duda. Con `KFold estándar`, la métrica es **indefinida (`NaN`)** en 8 de las 10 iteraciones, porque algunos folds quedan vacíos de ejemplos positivos. Solo por azar, en 2 iteraciones se obtiene un valor numérico ($\approx$ 0,49), que además es desfavorable (cercano a una clasificación aleatoria). Con `StratifiedKFold`, la AUC-ROC es siempre computable: media = 0,5283, desv. típica = 0,0253, en las 10 iteraciones.
- **La AUC-ROC reafirma empíricamente la necesidad de estratificar:** el hecho de que `KFold` haga la métrica indefinida en el 80 % de los experimentos no es un artefacto numérico, sino la *manifestación directa* del problema demostrado matemáticamente en la Tarea 1 mediante la distribución hipergeométrica: cuando los folds no están estratificados, es probable que algunos queden sin representantes de la clase positiva, haciendo imposible cualquier evaluación orientada a esa clase. La AUC-ROC, al ser la métrica más exigente en ese sentido, lo pone de manifiesto de forma inequívoca.

1.4. Tarea 4 — Detección de *data leakage*

1.4.1. Comparativa: con y sin variable trampa

Escenario	Accuracy	F1-macro
Sin variable trampa (*evaluación real*)	0,5850	0,3887
Con variable trampa (`ID_Hospital_Filtro`)	**1,0000**	**1,0000**

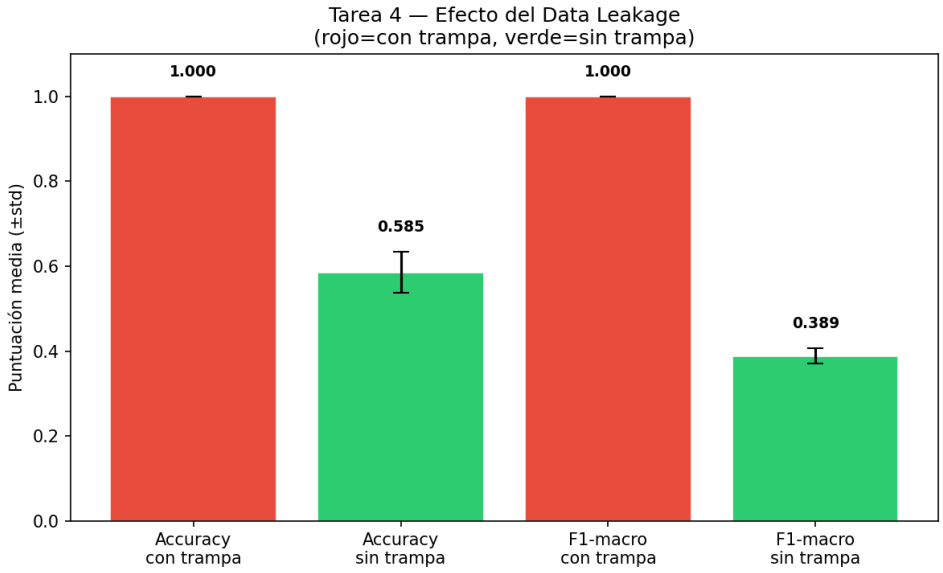

Figura 1.4: Efecto del *data leakage*. Las barras rojas corresponden al escenario con la variable trampa; las verdes, al escenario real sin ella.

Preguntas:

- *¿Qué variable está filtrando información del objetivo?* La variable `ID_Hospital_Filtro`.
- *¿Por qué esa variable invalida la evaluación?* Ha sido construida con una correlación perfecta (o casi perfecta) con la clase objetivo. El modelo aprende a usar este "filtro" en lugar de las variables clínicas reales, obteniendo un rendimiento del 100 % que es imposible en la práctica real.
- *¿Diferencia entre un modelo "bueno" y uno contaminado por leakage?* A diferencia de un modelo legítimo, uno con leakage simplemente está haciendo trampa usando información que no estará disponible en producción, por lo que su 100 % de acierto es un espejismo.

1.5. Reto — Auditoría metodológica

1. **Partición aleatoria:** Como hemos comprobado, corremos el riesgo de dejar el test sin apenas casos positivos, lo que invalida cualquier métrica que intentemos calcular.

2. **Estratificación:** Es la solución directa al problema anterior, ya que nos garantiza una representación mínima de la clase minoritaria en cada fold para poder evaluar de forma justa.

3. **Fuga de datos (Leakage):** Invalida por completo el experimento. El modelo no aprende, solo usa la variable trampa como atajo, por lo que fallará estrepitosamente en el mundo real.

4. **Buenas prácticas:** Usar siempre estratificación, auditar variables en busca de correlaciones sospechosas, emplear métricas robustas (F1, AUC-ROC) y documentar la reproducibilidad mediante semillas y validación cruzada repetida.

1.6. Conclusión final

Esta práctica me ha dejado claro que evaluar un modelo no es solo ejecutar unas líneas de código y mirar la *Accuracy*: con datos desbalanceados, hay que tomar decisiones de diseño concretas o los números que obtenemos nos van a engañar.

En cuanto a la **partición**, calculé con la distribución hipergeométrica que hay un $P(X = 0) \approx 1{,}10\,\%$ de probabilidad de que el test quede sin ningún positivo, y casi un 7 % de terminar con uno solo. En esas condiciones, F1-score, Recall y AUC-ROC son directamente indefinidas. Eso ya es razón suficiente para usar siempre estratificación: no es opcional si queremos que los resultados tengan sentido.

Los resultados obtenidos con 10 iteraciones de validación cruzada lo confirman: con `KFold` estándar, la AUC-ROC fue `NaN` en 8 de las 10 iteraciones porque algunos folds se quedaron sin positivos; con `StratifiedKFold` siempre fue computable y más estable (*std* menor en las tres métricas).

En cuanto al **data leakage**, el caso de `ID_Hospital_Filtro` es bastante claro: una variable artificialmente correlacionada con el target hace que el modelo alcance un 100 % de Accuracy, lo que en la práctica significaría entregar un sistema que falla en cuanto se usa con datos reales. La regla práctica que saco de aquí es auditarlas siempre antes de entrenar.